

11. Un octet s'arrête à 255

Après 255 vient 0, et avant 0 vient 255 : la variable fait le tour.

1. Ce qu'il faut comprendre

Un octet, ce sont **8 bits** : $2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 256$ valeurs, de 0 à 255. Il n'y a pas de 256 : la case n'a pas la place de l'écrire.

`250 + 10` devrait faire 260. La case garde **$260 - 256 = 4$** . On dit qu'elle **déborde** — comme un compteur kilométrique qui repasse à zéro. Et dans l'autre sens, `3 - 5` ne donne pas -2 mais **254** : il n'y a pas de nombres négatifs dans un `uint8_t`.

Ce n'est pas une panne, c'est la règle — et beaucoup de bogues de jeux Game Boy viennent de là : un personnage à $x = 0$ qui recule d'un pas se retrouve à $x = 255$, tout à droite. **Avant de retirer, on vérifie qu'il reste de quoi retirer** — ce sera le travail des conditions, au chapitre suivant.

Le compteur du bas avance d'un à chaque tour de boucle : après 255, il repasse par 000. Regarde-le faire le tour.

À toi : que vaut `0 - 1` ? Et `128 + 128` ? Devine d'abord, puis affiche-les.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **752 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         98 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        368 o ← CopierTuiles - 23 lettres sur 43
"250 PLUS 10:" 12 o ← texte() - un texte gravé
"3 MOINS 5:"   10 o ← texte() - un texte gravé
"COMPTEUR:"    9 o ← texte() - un texte gravé
```

Lettres gravées : CEILMNOPRSTU0123456789:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 3    33 o sur place, dans main() + EcrireTexte (10 o)
```

```
nombre() × 3    30 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : montre qu'une case uint8_t ne va que de 0 à 255.
// Au-dessus de 255, elle repart de 0 ; en dessous de 0, elle repart de 255.
// On dit qu'elle « déborde », comme un compteur kilométrique.
```

```
#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    uint8_t score = 250;    // Une case « score » qui contient 250.
    score = score + 10;     // 250 + 10 devrait faire 260, mais 260 ne tient pas :
                            // la case garde 260 - 256 = 4.

    uint8_t vies = 3;       // Une case « vies » qui contient 3.
    vies = vies - 5;        // 3 - 5 devrait faire -2, mais il n'y a pas de nombres
                            // négatifs dans un uint8_t : la case garde 256 - 2 = 254.

    texte(1, 2, "250 PLUS 10:"); nombre(15, 2, score); // affiche 004
    texte(1, 4, "3 MOINS 5:");   nombre(15, 4, vies);   // affiche 254

    uint8_t compteur = 0;    // Un compteur qui commence à 0.
    texte(1, 9, "COMPTEUR:"); // Son nom, écrit une seule fois avant la boucle.

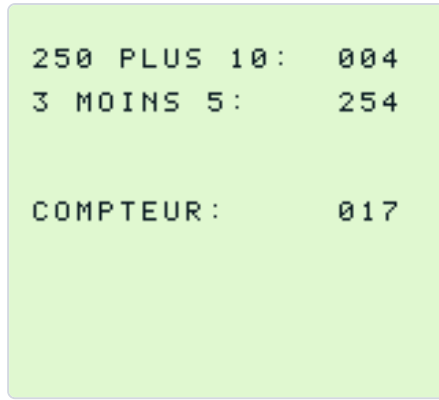
    while (true) {          // Répète pour toujours :
        image();            // attend l'image suivante (60 fois par seconde),
        compteur++;         // « ++ » ajoute 1 : c'est la même chose que
                            // compteur = compteur + 1.
                            // On voit : ... 253, 254, 255, 0, 1, 2 ...
        nombre(15, 9, compteur); // réécrit la valeur à chaque image.
    }
}
```

3. Ce qui se passe, étape par étape

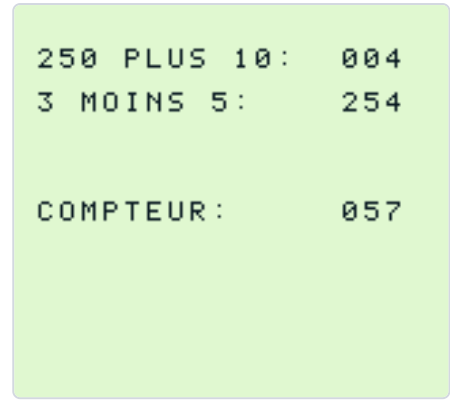
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



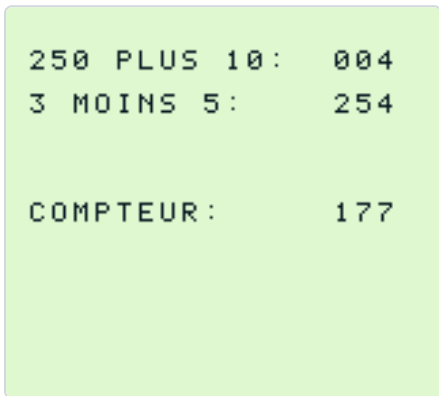
la toute première image — la console vient de démarrer — image 1



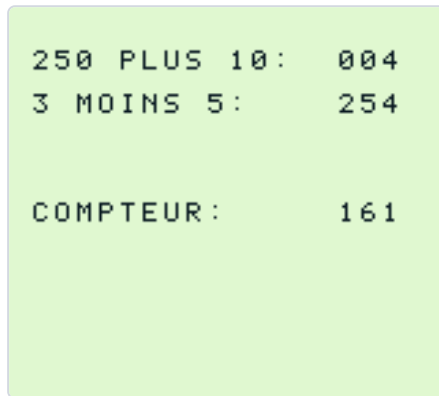
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

250 + 10 affiche 004, 3 - 5 affiche 254, et le compteur tourne de 000 à 255 puis recommence.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ 250 + 10 garde 4
- ✓ 3 - 5 garde 254
- ✓ le compteur passe de 255 à 0 (255 puis 0)

Ensuite : **12. Où vit une variable : avant la boucle, ou dedans** — chapitre 2 : Les variables : la mémoire de la console. · gameboy3 — écrire une cartouche Game Boy en C++